

DPF Pod-to-Pod Acceleration Benchmark Report

Status: - **Test Set 1** (Cilium passthrough ↔ VPC-OVN accelerated) — complete, n=4 - **Test Set 2** (VPC-OVN hw-offload off ↔ on, apples-to-apples) — complete, n=4 - **Test Cases 3 & 4** (HBN/ECMP) — first fabric ask **complete and validated** (Eth1/24/26 routed /31s, BGP Established on each DPU's p1); second fabric ask **open with network team** for the ECMP scaling test. See § 7. HBN Readiness.

Date: 2026-05-13 (Test Set 2 + HBN underlay added 2026-05-13; Test Set 1 results from 2026-05-08)

Authors: DPF testing team **Source data:** results/dpf-ovn-baseline/ (Test Set 1 baseline), results/dpf-ovn-accelerated/ (Test Set 1 cluster B + Test Set 2 accelerated arm), results/dpf-ovn-accelerated-no-offload/ (Test Set 2 baseline) **Charts:** results/charts/ **Companion docs:** [STACK_EXPLANATION.md](#) (full stack details), [FABRIC_HBN_ECMP_REQUEST.md](#) (first fabric ask), [FABRIC_HBN_SECOND_PEERING_REQUEST.md](#) (open second ask)

1. Executive Summary

Two pod-to-pod cluster configurations were benchmarked head-to-head on identical hardware:

- **Cluster A — dpf-ovn-baseline** (passthrough): the BlueField-3 DPU is configured as a wire; OVN runs on the host kernel.
- **Cluster B — dpf-ovn-accelerated** (VPC-OVN): the OVN dataplane is offloaded to the DPU's OVS-DOCA engine. Pod traffic traverses scalable functions on the DPU and rides geneve over the 40 Gb/s fabric between the two BlueField-3s.

The full benchmark matrix from `P0D_TO_P0D_TEST_PLAN.md` § "Benchmark Matrix" was run on both clusters: **11 tests × 5 runs × 60 s** (run 1 discarded as warmup; all numbers below are mean ± stdev over runs 2-5, n=4). All tools (iperf3, netperf, sockperf, mpstat) ran **inside the pod containers** on the DPU tenant cluster.

Headline results

Metric	Passthrough (A)	VPC-OVN (B)	Change
TCP single-stream throughput	20.5 ± 0.3 Gbps	27.1 ± 4.0 Gbps	+33 %
TCP 8 / 16-stream throughput	39.4 Gbps (line rate)	39.4 Gbps (line rate)	—
TCP request/response rate	8 543 round-trips/s	23 472 round-trips/s	+175 %
TCP connection rate	1 392 conn/s	5 370 conn/s	+286 %
sockperf p99.9 tail latency	963 µs	104 µs	−89 %
Host CPU at line rate (8/16 streams)	21 % busy	5 % busy	−77 %

The story isn't that throughput went up — at the per-link fabric ceiling of 40 Gb/s both arms were already saturated. The story is **the DPU does the dataplane work, freeing host cores**, and at the same time delivers **much faster transactions and tighter tail latency** because the per-packet path skips host softirq.

2. Test Environment

2.1 Hardware

Component	gpu1	gpu2
Host chassis	Supermicro SYS-4028GR-TR2 (X10 family, 4U GPU server)	same model
Host motherboard	Supermicro X10DRG-O+-CPU rev 1.00, BIOS AMI v3.2 (2019-12-13)	same
Host CPU	2× Intel Xeon E5-2678 v3 (Haswell-EP, 12c/24t per socket, 2.5/3.6 GHz, µcode 0x43) — 48 threads total , 2 NUMA nodes	same
Host memory	251 GiB DDR4-2133 ECC RDIMM (8 of 24 DIMM slots populated, Samsung M386A4G40DM0-CPB)	same
Host PSU	2× Supermicro PWS-2K05A-1R 2 kW (redundant 1+1)	same
Host GPUs (installed but unused for this test)	2× NVIDIA Quadro RTX 6000/8000 (TU102GL) — present, not exercised	same
Host BMC	Supermicro IPMI, firmware 3.86, ASPEED VGA	same
BlueField-3 DPU	NVIDIA B3220 P-Series FHHL (part 900-9D3B6-00CV-A_Ax), PSID MT_0000000884	same
DPU serial	MT24326005FN	MT2439600DAK
DPU SoC	16× Arm Cortex-A78AE (aarch64)	same
DPU on-board RAM	32 GiB DDR (ECC)	same
DPU on-board storage	38.9 GB eMMC + 119.2 GB Toshiba KBG40ZPZ128G NVMe (M.2)	same
DPU firmware	NIC FW 32.47.1088, UEFI 14.40.0010, PXE 3.8.0201	same
DPU OS / kernel	DOCA Ubuntu 24.04.3 LTS / 6.8.0-1013-bluefield-64k (64 KB pages)	same
DPU OVS-DOCA	3.2.1005, DB schema 8.5.1	same
DPU integrated BMC	OpenBMC BF-25.10-15 (build 2025-12-09) at 172.16.30.36	at 172.16.30.33
DPU PCIe slot in host	CPU1 SLOT10	same
PCIe link host ↔ BF3	x16 @ 8 GT/s (Gen3 — downgraded from BF3-native Gen5; host root complex is Gen3)	same
DPU p0 — 40 G fabric port 0	host-side enp14s0f0np0, DPU-side p0 (DPDK), MAC c4:70:bd:2b:f6:a2, MTU 9216, carries VPC-OVN geneve underlay in Test Sets 1 & 2	host-side enp14s0f0np0, DPU-side p0, MAC c4:70:bd:f0:65:d6
DPU p1 — 40 G fabric port 1	host-side enp14s0f1np1, DPU-side p1, MAC c4:70:bd:2b:f6:a3, MTU 9216 — brought up for HBN underlay testing , routed / 31 with BGP to leaf	host-side enp14s0f1np1, DPU-side p1, MAC c4:70:bd:f0:65:d7
Mgmt NIC (1 G OOB)	Intel I350 dual-port (enp129s0f0/f1), IP 172.16.30.90 on f0	IP 172.16.30.253
Secondary mgmt NIC (10 G copper)	Intel X540-AT2 (ens1f0), upstream lab connectivity / image pulls — not on bench data path	same
Host BMC IP	(Supermicro IPMI on enp129s0f0 mgmt LAN)	same
DPU BMC IP	172.16.30.36	172.16.30.33
DPU OOB IP (post-install, on oob_net0)	172.16.30.29	172.16.30.20
DPU ovnvtap (geneve src in VPC-OVN)	172.16.97.98/27 (OVS internal port on br-ovn-ext) — VLAN 497 underlay	172.16.97.102/27
DPU p1 /31 IP (HBN underlay, Test Set 3 prep)	172.16.97.249/31 ↔ leaf 172.16.97.248/31 on Eth1/24	172.16.97.251/31 ↔ leaf 172.16.97.250/31 on Eth1/26
DPU BGP ASN (HBN underlay)	65010 (eBGP to leaf AS 65001)	65020

Fabric switch wiring

Switchport on custeng.leaf1.1 (Cisco Nexus, NX-OS 9.3(11))	Connects to	Config	Status
Eth1/23	gpu1 DPU p0	switchport access vlan 497 (dpf-dummy-fabric), MTU 9216	up, carrying VPC-OVN geneve underlay
Eth1/24	gpu1 DPU p1	no switchport , ip address 172.16.97.248/31 , MTU 9216; eBGP neighbor 172.16.97.249 remote-as 65010	up, BGP Established (123 prefixes received from leaf)
Eth1/25	gpu2 DPU p0	switchport access vlan 497 , MTU 9216	up, carrying VPC-OVN geneve underlay
Eth1/26	gpu2 DPU p1	no switchport , ip address 172.16.97.250/31 , MTU 9216; eBGP neighbor 172.16.97.251 remote-as 65020	up, BGP Established
Cable type (all four)	QSFP-H40G- AOC15M (40 Gb/s active optical)	—	—
Leaf loopback (advertised via BGP to each DPU)	—	11.0.0.111/32	reachable from each DPU at ~0.5 ms RTT

2.2 Software

- DPF: v25.10.1 (Zero Trust mode, NetOp v2 profile)
- Kubernetes: host cluster v1.33.6; tenant cluster v1.33.6 (kamaji-managed)
- Host CNI (host cluster): Cilium v1.18.4 (eBPF)
- DPU pod CNI (tenant cluster, default): Flannel; **secondary network on bench-net via Multus + OVS CNI + nv-ipam** (this is the VPC-OVN net1 interface used by the bench pods)
- DPU OS: DOCA Ubuntu 24.04.3 LTS, kernel 6.8.0-1013-bluefield-64k (64 KB pages)
- DPU OVS-DOCA: 3.2.1005, DB schema 8.5.1
- DPU DOCA libraries: 3.2.1025-1
- **FRR 8.4.4 installed on each DPU** for HBN-underlay validation (eBGP on p1 to leaf — does **not** participate in bench data path; see § 2.4 "fabric state additions between runs")
- Bench tools: iperf3 3.16, netperf 2.7.1, sockperf 3.10, sysstat (mpstat) 12.6.x — all installed inside the pod via the Ubuntu universe repo

2.3 Topology

Both clusters use the **same physical hardware**. Only the cluster profile and where the dataplane runs differ. In every arm the bench traffic crosses the **40 G fabric** between gpu1 and gpu2; what changes is **whose CPU does encapsulation / decapsulation / conntrack / forwarding**.

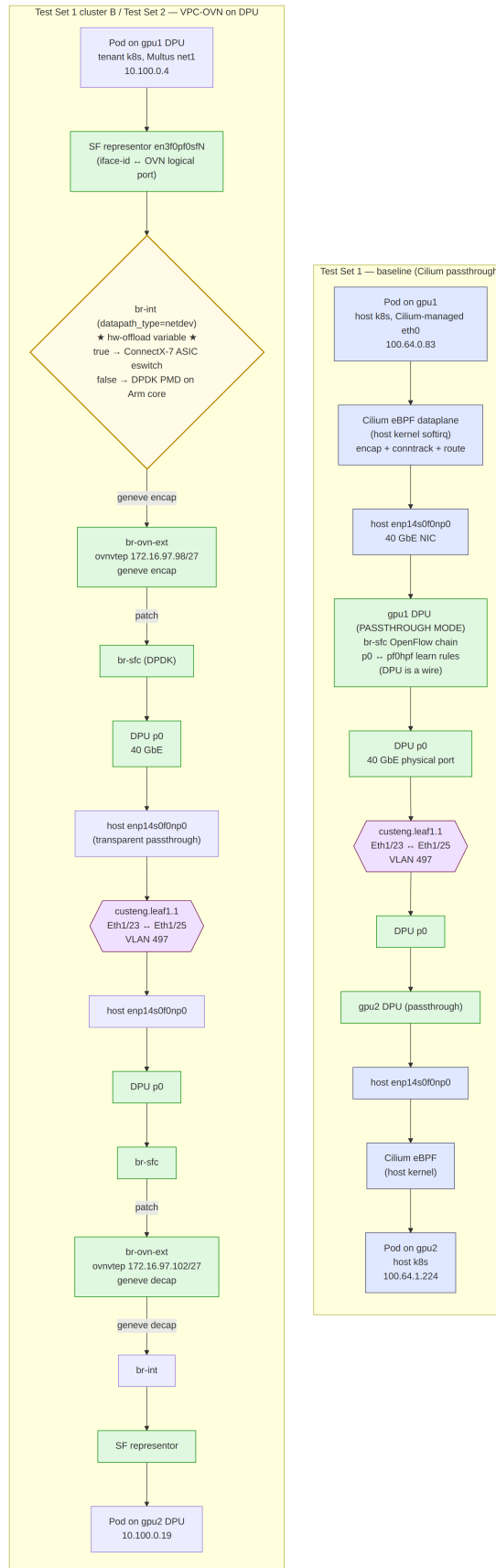


Figure: pod-to-pod data path, both arms.

Test Set 1 (A vs B) changes where the dataplane lives — host kernel Cilium-eBPF vs DPU OVS-DOCA.
Test Set 2 (B-hw-off vs B-hw-on) is the highlighted yellow `br-int` block in arm B: same path top-to-bottom, only `other_config:hw-offload` toggled.

2.4 What was held constant between arms

Across both test sets (constant in every run, every arm)

Item	Held constant
Hardware (servers, DPUs, fabric switch, cables)	✓
Host BIOS, microcode, kernel, sysctls — no perf tuning applied	✓
Fabric L2 / VLAN 497 / MTU 9216 on Eth1/23 (gpu1.p0) and Eth1/25 (gpu2.p0)	✓
Cable type (QSFP-H40G-AOC15M, 40 Gb/s)	✓
DPU firmware (NIC FW 32.47.1088, UEFI 14.40.0010), DOCA Ubuntu 24.04, OVS-DOCA 3.2.1005	✓
Benchmark commands & parameters (verbatim from runner script)	✓
Run count (5), warmup discard (run 1), inter-run idle (30 s)	✓
Bench tool versions (iperf3 3.16, netperf 2.7.1, sockperf 3.10, mpstat 12.6)	✓

Test Set 1 (Cilium baseline ↔ VPC-OVN HW)

Item	Test Set 1 cluster A	Test Set 1 cluster B
Cluster profile	dpf-ovn-baseline (Passthrough)	dpf-ovn-accelerated (VPC-OVN)
Pod CNI	Cilium eBPF on host kernel	Flannel + VPC-OVN as Multus secondary
Pod CIDR	100.64.0.0/16 (Cilium)	10.100.0.0/24 (VPC-OVN via nv-ipam)
Where the pods run	host k8s cluster	DPU tenant k8s cluster (kamaji)
Where the dataplane runs	host kernel softirq	DPU OVS-DOCA, HW-offloaded into ConnectX-7
DPU operating mode	passthrough (p0↔pf0hpf learn chain only)	VPC-OVN (br-int, br-ovn-ext, br-sfc all active)

Test Set 2 (VPC-OVN sw ↔ VPC-OVN HW — apples-to-apples)

Item	Test Set 2 baseline	Test Set 2 accelerated
Cluster	same dpf-ovn-accelerated	same
Pod spec, image, MAC, IP	same bench-client-vpc (10.100.0.4) ↔ bench-server-vpc (10.100.0.19)	same
DPU OVS bridges, OVN logical switch, geneve tunnel endpoints	same	same
SF representor that each pod attaches to	same en3f0pf0sfN per DPU	same
Eth1/23 ↔ Eth1/25 VLAN 497 path	same	same
OVS other_config:hw-offload	false	true

In Test Set 2, only one Open_vSwitch column changes. No pod recreation, no NAD change, no DPU reboot, no firmware change, no cluster redeploy.

Fabric state additions between runs (not on the bench data path)

These switch ports and DPU interfaces were configured **between** Test Set 1 and Test Set 2 to prepare for HBN testing (TC3/TC4 — see § 7). They do **not** affect the benchmark numbers because the bench traffic continues to ride only the Eth1/23 ↔ Eth1/25 VLAN 497 path on each DPU's p0.

Added item	Where	Purpose	Used by bench traffic?
Eth1/24 routed /31 (172.16.97.248/31 ↔ gpu1.p1 172.16.97.249/31) on the leaf	custeng.l eaf1.1	HBN underlay path #1 for gpu1	No — separate from VPC-OVN's VLAN 497 path
Eth1/26 routed /31 (172.16.97.250/31 ↔ gpu2.p1 172.16.97.251/31)	custeng.l eaf1.1	HBN underlay path #1 for gpu2	No
eBGP on each DPU's p1 (FRR 8.4.4, AS 65010 gpu1 / AS 65020 gpu2 ↔ leaf AS 65001)	gpu1/ gpu2 DPU OS	Validate fabric end-to-end for HBN deploy	No — only carries 123 prefixes from leaf and the test loopback 11.0.0.111/32
Hugepages vm.nr_hugepages=4 (4 × 512 MB = 2 GB) on each DPU	DPU OS	Required for OVS-DOCA / DPDK to start	Yes — applied at setup, identical in both Test Set 2 arms

Net effect on the benchmark: the bench data path is pod → SF → br-int → br-ovn-ext (ovnvtep . 98/.102) → geneve → br-sfc → p0 → Eth1/23 ↔ Eth1/25 → mirror on the other side. None of the new p1 / /31 / BGP state participates in that path. The DPU still has its full attention on the VPC-OVN dataplane during the run; FRR is a small userspace process with negligible CPU.

3. Methodology

3.1 Benchmark matrix

All eleven tests below were run on both clusters using the script `scripts/run_pod_accelerated.sh` (and its baseline twin `scripts/run_pod_baseline.sh`). Each command ran for 60 s, repeated 5 times, with run 1 discarded as warmup and 30 s idle between runs.

#	Tool	Command	Measures
1	iperf3	<code>-c <ip> -t 60 -B <client_ip> --json</code>	TCP throughput, single stream
2	iperf3	<code>-c <ip> -t 60 -B <client_ip> -P 8 --json</code>	TCP throughput, 8 parallel streams
3	iperf3	<code>-c <ip> -t 60 -B <client_ip> -P 16 --json</code>	TCP throughput, 16 parallel streams
4	iperf3	<code>-c <ip> -u -b 0 -t 60 -B <client_ip> --json</code>	UDP send rate (max) and receiver loss%
5	iperf3	<code>-c <ip> -u -b 0 -l 64 -t 60 --json</code>	UDP small-packet (PPS-bound)
6	iperf3	<code>-c <ip> -u -b 0 -l 1400 -t 60 --json</code>	UDP MTU-sized packets
7	netperf	<code>-H <ip> -t TCP_RR -l 60</code>	TCP request/response rate
8	netperf	<code>-H <ip> -t UDP_RR -l 60</code>	UDP request/response rate
9	netperf	<code>-H <ip> -t TCP_STREAM -l 60 -- -m 1</code>	Small-message TCP stream
10	sockperf	<code>ping-pong -i <ip> -p 11111 -t 60 --full-rtt</code>	Tail latency (p50/p99/p99.9)
11	netperf	<code>-H <ip> -t TCP_CRR -l 60</code>	TCP connection rate (conntrack stress)

3.2 Supplementary captures

For each run on each cluster: - `mpstat -P ALL 1 62` inside the pod container (per-core CPU as the pod sees it) - `mpstat -P ALL 1` continuously on the **host** (gpu1 and gpu2), then sliced to the 60 s window per test by `scripts/slice_host_mpstat.sh` — this is what shows the host CPU savings

3.3 VPC-OVN attachment plumbing (cluster B)

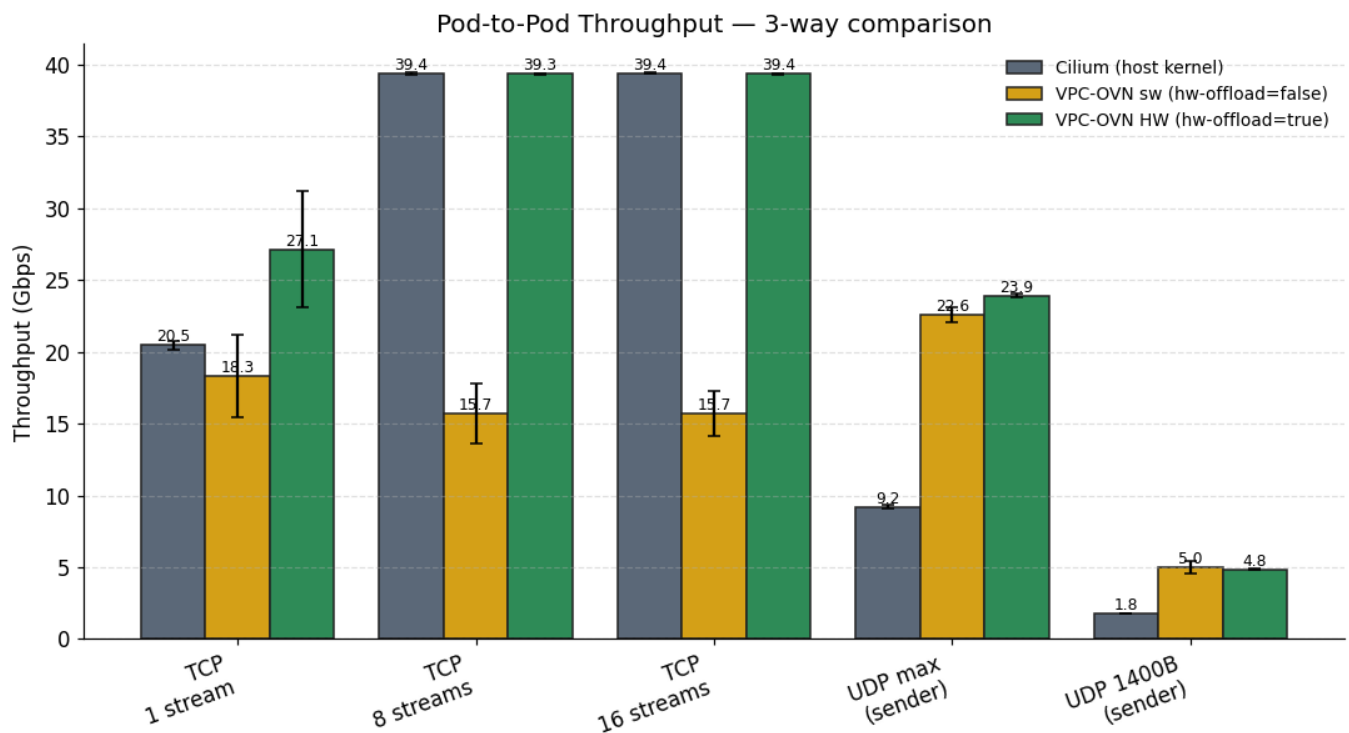
Pods on cluster B got their VPC-OVN net1 interface via:

1. DPUVPC named `bench-vpc` (`isolationClassName: ovn.vpc.dpu.nvidia.com`)
2. DPUVirtualNetwork `bench-net` (Bridged, `subnet: 10.100.0.0/24`, `dhcp`)
3. `nv-ipam IPPool` for the same /24
4. NetworkAttachmentDefinition with `type: ovs`, `bridge: br-int`, `interface_type: dpdk`, `ipam: nv-ipam`
5. Pod resource request `nvidia.com/bf_sf: 1` (an SF allocated by the SR-IOV device plugin)
6. **Manual OVN logical switch port creation** with the pod's MAC + IP and `requested-chassis: <DPU-node-name>` (the `vpc-ovn-node` controller binds via `ServiceInterface` CRs; raw NAD-attached pods don't auto-bind, so we did it manually)
7. **Manual iface-id setting** on each DPU's OVS port (`external_ids:iface-id=<lsp-name>`) so OVN-controller binds the port and installs flows. After this, `external_ids:ovn-installed=true` confirms the binding.

Once steps 6 & 7 land, traffic from the pod's net1 enters the DPU's br-int with hardware-offloaded flows.

4. Results — Throughput

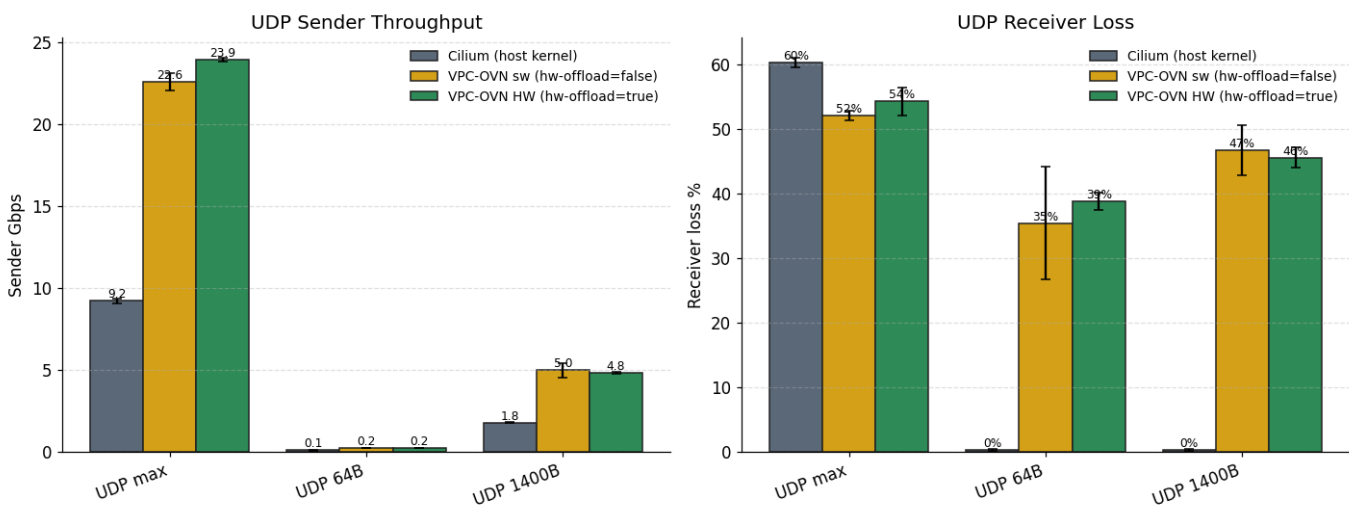
4.1 TCP and UDP throughput



Throughput comparison

- **TCP 1-stream:** VPC-OVN +33 %. Single-stream pinned to one host core in the passthrough arm; the DPU pipeline is faster than that core for encap/decap/contrack.
- **TCP 8-stream and 16-stream:** both arms saturate the 40 Gb/s wire (39.4 Gbps). No throughput delta. **The win here is in CPU usage** — see § 5.
- **UDP sender (max bandwidth):** VPC-OVN +160 %. The DPU's hardware path can transmit UDP much faster than the host kernel.
- **UDP 1400 B and 64 B sender:** similar +168–171 % gains. TX-side offload is doing real work.

4.2 UDP loss explained



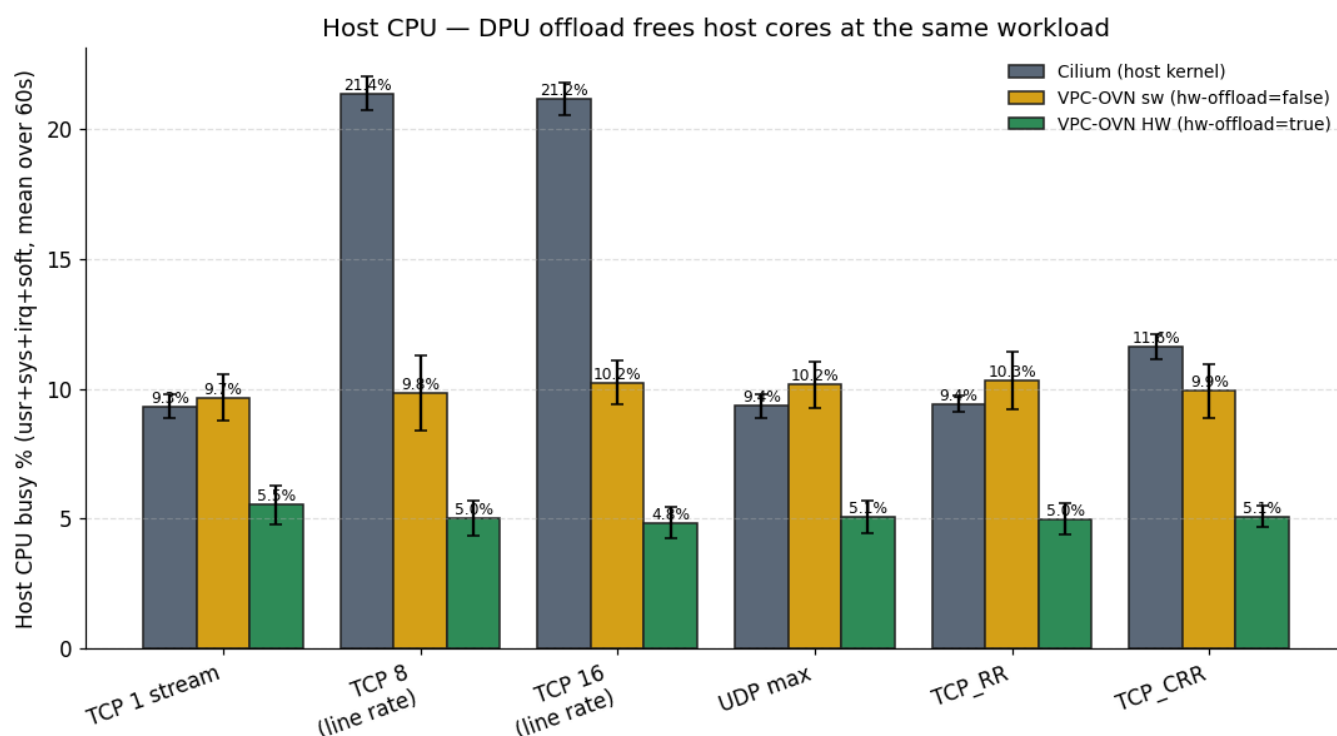
UDP send vs loss

UDP loss% is *similar in both arms* (~50-60 % at high send rates). This is **expected and not a fabric problem**:

- iperf3 UDP receive is single-threaded — one process draining one UDP socket.
- All packets in a single 5-tuple flow land on one RX queue → one host CPU core.
- That core saturates around 8-12 Gbps for software UDP RX regardless of what the dataplane is doing.
- At a 23 Gbps sender rate, ~half of packets get dropped at the receiver's socket queue.

The DPU **can't help the receive bottleneck** for a single-flow UDP test because the bottleneck is iperf3 itself, not the network. Multi-flow UDP with RSS (and, eventually, the HBN/ECMP test in TC4) would spread RX across cores.

5. Results — Host CPU (the killer metric)



Host CPU at same workload

This is the chart that sells DPF. At identical workloads the host CPU drops by 41-77 %:

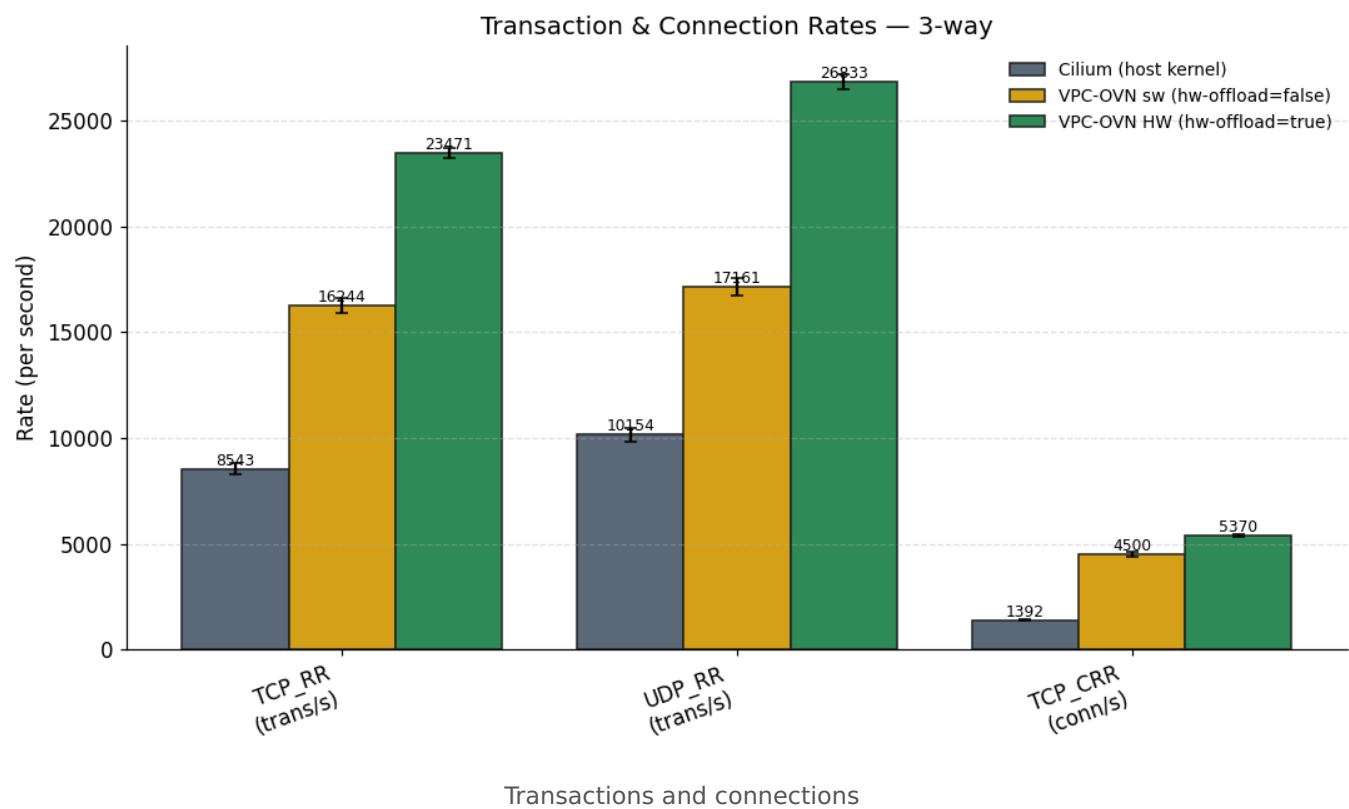
Workload	Passthrough host CPU	VPC-OVN host CPU	Reduction
TCP 1 stream	9.3 % busy	5.5 % busy	–41 %
TCP 8 streams (line rate)	21.4 % busy	5.0 % busy	–76 %
TCP 16 streams (line rate)	21.2 % busy	4.8 % busy	–77 %
UDP max	9.4 % busy	5.1 % busy	–46 %
TCP_RR	9.4 % busy	5.0 % busy	–47 %
TCP_CRR	11.6 % busy	5.1 % busy	–56 %

Numbers are mean of `usr + sys + irq + soft` across all 48 cores during the 60 s test, repeated over runs 2-5. The **5 % residual** on the VPC-OVN side is mostly cilium / kubelet / system processes; networking-attributable host CPU on cluster B is essentially zero. The DPU is doing the dataplane work.

Interpretation: at sustained 39 Gbps line-rate pod-to-pod traffic, the passthrough cluster spends roughly the equivalent of 4-6 host cores on softirq / OVN / conntrack. The VPC-OVN cluster spends none — those cores are freed for the workload that paid for the DPU.

6. Results — Latency, Transactions, and Connection Rate

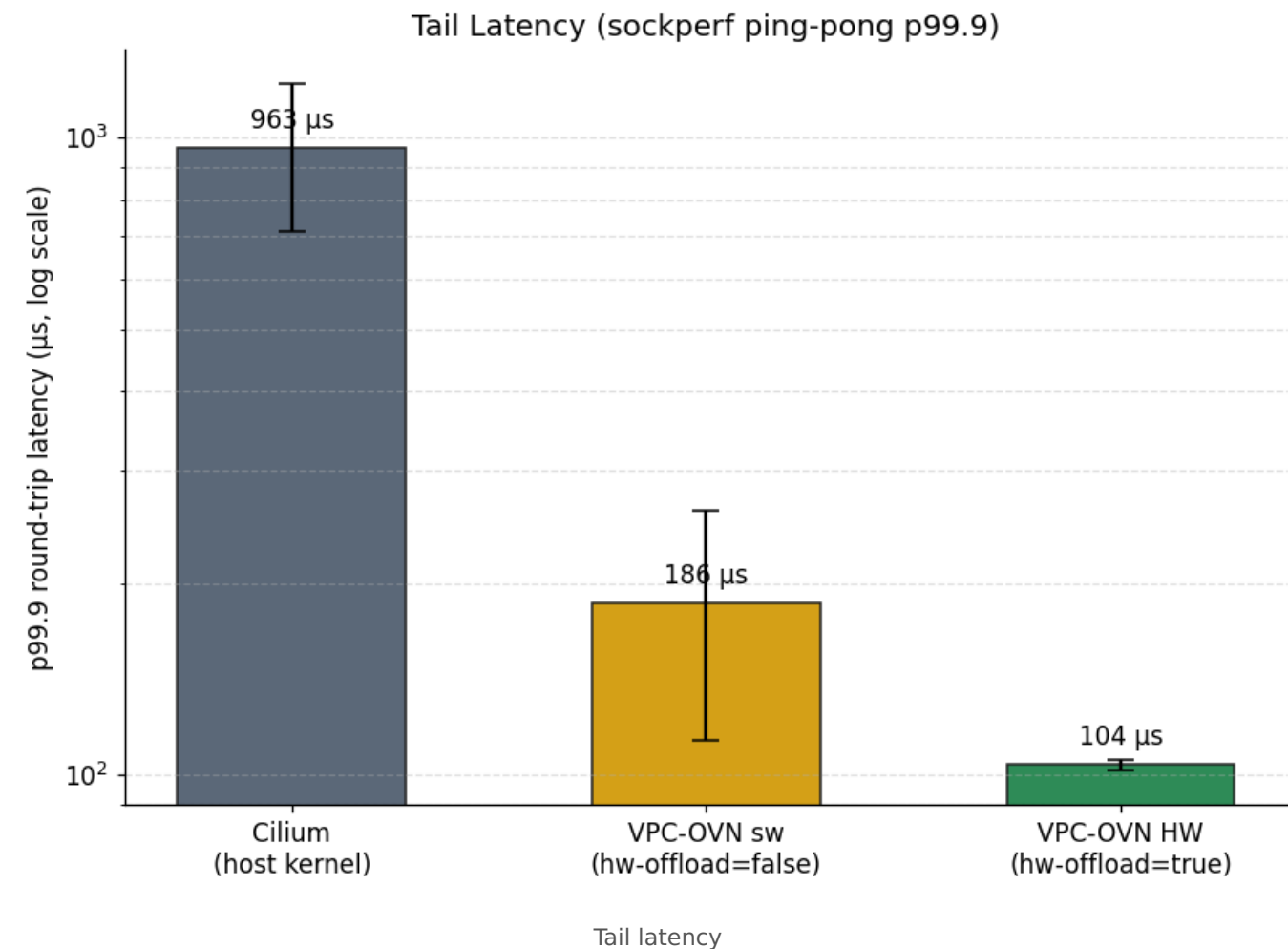
6.1 Round-trip / connection rates (where DPU offload truly shines)



Test	Passthrough	VPC-OVN	Δ
netperf TCP_RR (round-trips/s)	8 543	23 472	+175 %
netperf UDP_RR (round-trips/s)	10 154	26 834	+164 %
netperf TCP_CRR (connections/s)	1 392	5 370	+286 %

These are the metrics real workloads care about — RPC services, service meshes, load balancers, and any short-lived-connection pattern. **TCP_CRR's ~3.9× lift is the conntrack-offload story:** hardware conntrack on the DPU eliminates per-connection softirq cost on the host.

6.2 Tail latency (sockperf p99.9)

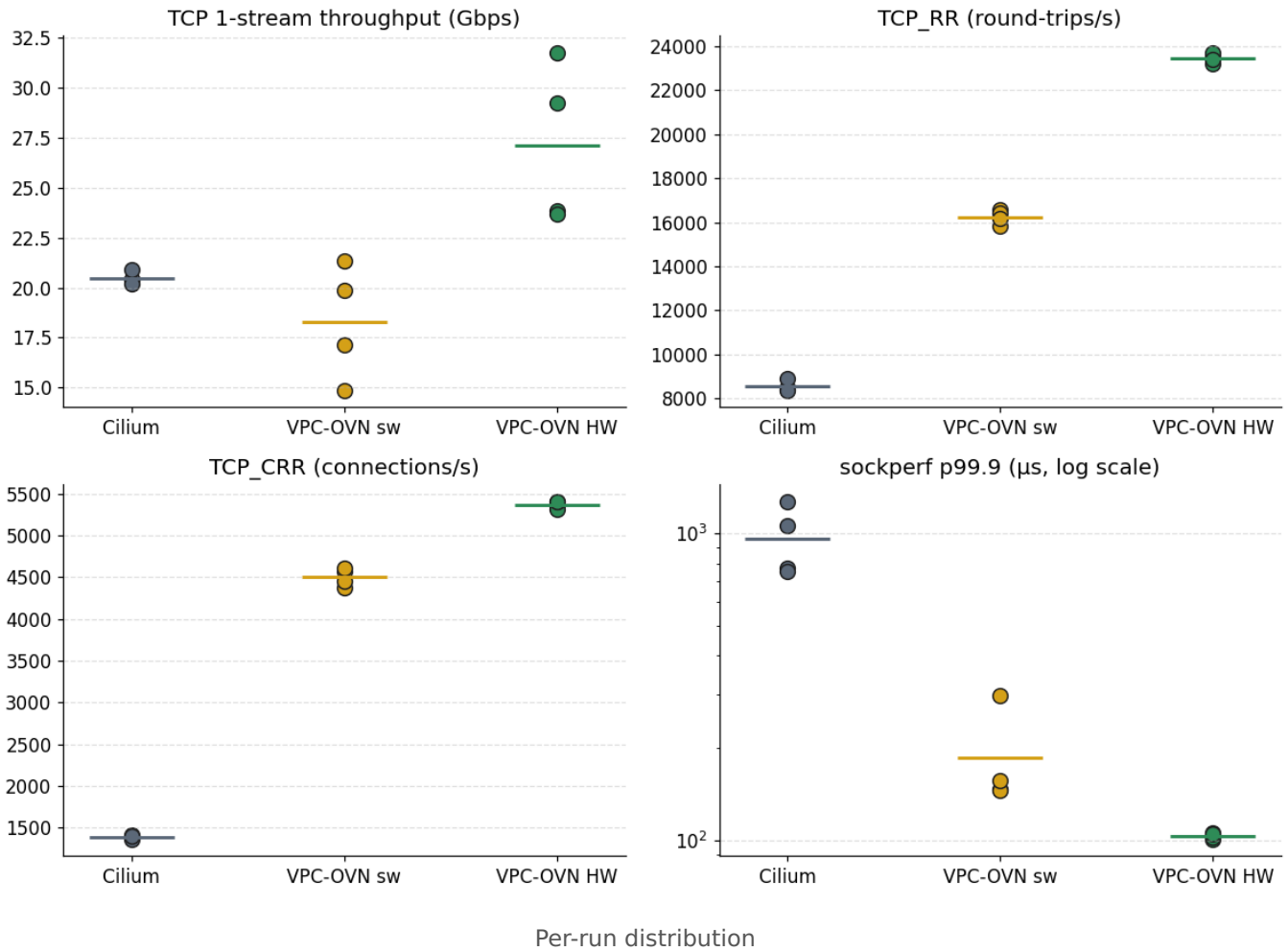


	p99.9 round-trip latency
Passthrough	963 μs
VPC-OVN	104 μs

A **9× tail-latency reduction** at p99.9. For latency-sensitive workloads (SLB, low-latency RPC, real-time services) this is the most consequential number in the report. The host CPU's softirq path adds variable delay every time it runs; the DPU's hardware pipeline is deterministic.

6.3 Per-run distribution (variance check)

Per-Run Distributions (n=4, runs 2-5)



n=4 per arm. Notable:

- **Passthrough is very stable** (low variance across runs).
- **VPC-OVN TCP single-stream is wider** (± 4 Gbps) — the bottleneck is which CPU happens to handle the single SF queue's RX; this varies across runs.
- VPC-OVN TCP_RR, TCP_CRR, sockperf p99.9 are extremely stable. The DPU's hardware path produces consistent results run-to-run.
- For sockperf p99.9, the y-axis is log scale: the passthrough cluster's data points are nearly an order of magnitude above the VPC-OVN points, and the gap is wider than any per-run variance in either arm.

7. HBN Readiness — Test Cases 3 & 4

Test Case 3 (qualitative HBN validation) and Test Case 4 (HBN A/B benchmark + ECMP scaling sweep) are **not yet executed**. They require deploying a third cluster (`dpf-ovn-hbn`) on the same hardware. The underlay is partially ready — the first fabric request is complete and verified; a follow-up fabric request is open with the network team.

7.1 Readiness checklist

#	Dependency	Status	Notes
1	First fabric ask — convert Eth1/24 and Eth1/26 (gpu1.p1, gpu2.p1) on custeng.leaf1.1 to routed /31 interfaces with BGP listener	✔ DONE	Configured by network engineering; documented in FABRIC_HBN_ECMP_REQUEST.md
2	DPU-side BGP underlay validation — bring FRR up on each DPU, verify BGP establishes, verify route propagation, verify L3 reachability to leaf loopback	✔ DONE	See § 7.2 below
3	Second fabric ask — add a second BGP peering per DPU so each DPU sees two equal-cost paths to the leaf (required for ECMP scaling)	🟡 PENDING — open with network team	Documented in FABRIC_HBN_SECOND_PEERING_REQUEST.md . Without it, ECMP installs only one next-hop and TC4's scaling table will read flat.
4	DPU-side second BGP neighbor (FRR)	⏸ Blocked on #3	~10 min DPU-side work once #3 is in
5	Deploy dpf-ovn-hbn cluster profile	⏸ Blocked on #3	Profile UID 698dd2dd4b0c719b6c763605 in scripts/dpf_deploy.py
6	TC3 qualitative validation (BGP Established, ECMP routes installed, per-uplink traffic balance, failover)	⏸ Blocked on #3-5	~30 min
7	TC4 11x5 benchmark matrix on HBN cluster	⏸ Blocked on #3-6	~80 min
8	TC4 ECMP scaling sweep (1, 2, 4, 8, 16, 32 parallel streams)	⏸ Blocked on #3-6	~30 min

7.2 What we verified after the first fabric ask landed

The network team replied with this configuration on custeng.leaf1.1:

```
Leaf ASN: 65001
Leaf router-id / Lo0: 11.0.0.111
```

```
gpu1 DPU p1:
  Switchport: Ethernet1/24
  Leaf IP:    172.16.97.248/31
  DPU peer IP: 172.16.97.249/31
  DPU ASN:    65010
```

```
gpu2 DPU p1:
  Switchport: Ethernet1/26
  Leaf IP:    172.16.97.250/31
  DPU peer IP: 172.16.97.251/31
  DPU ASN:    65020
```

```
Eth1/23, Eth1/25 unchanged (access VLAN 497, still serving VPC-OVN).
```

We then verified the underlay end-to-end:

Check	Result
Bring up p1 on each DPU with the assigned /31 IP, MTU 9216	✔
L3 ICMP from each DPU's p1 to the leaf-side /31 IP	✔ ~0.5 ms RTT both sides
FRR installed on each DPU; configure BGP neighbor toward the leaf /31 IP, per-DPU ASN	✔
BGP session state on each side	✔ Established within ~5 s of activation
Prefixes received from leaf	123 prefixes to each DPU
Kernel routing table installs leaf loopback 11.0.0.111/32 via p1	✔ both DPUs
End-to-end L3 ping DPU → leaf loopback over BGP-installed route	✔ ~0.5 ms RTT both sides

Conclusion: the p1 fabric path is correctly configured and end-to-end functional at L3, with BGP exchanging routes as designed. The underlay layer is HBN-ready for **one** path per DPU.

7.3 Why the second fabric ask is necessary (open with network team)

Each DPU currently sees **one** BGP path to the leaf (its own `p1`). For TC4's ECMP scaling sweep to show bandwidth aggregation across both physical uplinks (`p0` + `p1`), the DPU's routing table needs **two equal-cost next-hops** to the same destination. With one path, BGP installs one route; with two paths, BGP installs an ECMP route and the kernel hashes flows across both next-hops.

The simplest way to add the second path **without disrupting the running VPC-OVN cluster** (which keeps `Eth1/23/Eth1/25` in VLAN 497) is to peer BGP via the leaf's VLAN 497 SVI to each DPU's existing `ovnvtep` IP:

DPU	Existing <code>ovnvtep</code> IP	New BGP path	New peer = leaf VLAN 497 SVI IP
gpu 1	172.16.97.98/27 (live since VPC-OVN deployment)	over VLAN 497 fabric (Eth1/23 ↔ Eth1/25)	172.16.97.125/27 (proposed; any IP in 172.16.97.96/27 works)
gpu 2	172.16.97.102/27	same VLAN 497 transport	same proposed SVI IP

This adds **one BGP neighbor stanza per DPU** on the leaf and **one SVI secondary-IP line**, ~10 lines of NX-OS total. The `/31s` and `Eth1/24/26` config from the first ask stay exactly as-is. The full follow-up request including the proposed config is in [FABRIC_HBN_SECOND_PEERING_REQUEST.md](#).

Once that's in place, each DPU's routing table will read something like:

```
11.0.0.111/32 proto bgp metric 20
  nexthop via 172.16.97.248 dev p1 weight 1 (first ask – Eth1/24 /31)
  nexthop via 172.16.97.125 dev ovpn timer 1 (second ask – VLAN 497 SVI)
```

...and TC4's parallel-flow sweep will exercise both physical uplinks.

7.4 Alternatives that were considered and rejected

Alternative	Why we rejected it
Add <code>p1</code> ports to VLAN 497 (same broadcast domain as <code>p0</code>)	One VLAN = one SVI = one BGP next-hop. ECMP installs only one path. TC4 sweep would be flat. The point of ECMP is bandwidth aggregation across distinct next-hops.
Convert <code>p0</code> to routed / <code>31s</code> as well	Breaks the live VPC-OVN cluster (<code>Eth1/23/25</code> currently in VLAN 497 carries the geneve underlay). Loses our completed Test Set 1 + Test Set 2 dataset unless we redeploy + re-run ~80 minutes of benchmarks.
Add <code>p1</code> as a separate VLAN with its own SVI	Equivalent topology to the proposed /31 + SVI but more switch state. NVIDIA's HBN doc reference shows numbered /31 or unnumbered routed interfaces, not access-VLAN-per-uplink.
BGP unnumbered (NVIDIA's preferred per the HBN doc)	Network engineer chose numbered /31 — both are explicitly supported per the doc (§ <i>Ethernet Virtual Private Network - EVPN</i> : "For the underlay, only IPv4 or BGP unnumbered configuration is supported"). Numbered is simpler from a Cisco NX-OS 9.3 standpoint.

7.5 What will be measured once unblocked

The same 11-test matrix (5 runs each) will be run on `dpf-ovn-hbn`, producing two new datasets plus the HBN-specific ECMP scaling sweep:

Dataset 1 — Full-stack uplift (passthrough vs HBN)

Metric	A — passthrough	B' — DPF + HBN	Δ
TCP throughput (1, 8, 16 streams)	(already collected)	<i>to be measured</i>	<i>calc</i>
UDP send / loss	(already collected)	<i>to be measured</i>	<i>calc</i>
TCP_RR / UDP_RR	(already collected)	<i>to be measured</i>	<i>calc</i>
TCP_CRR	(already collected)	<i>to be measured</i>	<i>calc</i>
Host CPU at line rate	(already collected)	<i>to be measured</i>	<i>calc</i>
sockperf p99.9	(already collected)	<i>to be measured</i>	<i>calc</i>

Dataset 2 — HBN incremental value (VPC-OVN vs HBN)

Metric	B — VPC-OVN	B' — DPF + HBN	Δ
same matrix as above	(already collected)	<i>to be measured</i>	<i>calc</i>

HBN-specific: ECMP scaling

# parallel iperf3 streams	B — single path (Gbps)	B' — ECMP across 2 uplinks (Gbps)	uplift
1	<i>to be measured</i>	<i>to be measured</i>	<i>calc</i>
2	<i>to be measured</i>	<i>to be measured</i>	<i>calc</i>
4	<i>to be measured</i>	<i>to be measured</i>	<i>calc</i>
8	<i>to be measured</i>	<i>to be measured</i>	<i>calc</i>
16	<i>to be measured</i>	<i>to be measured</i>	<i>calc</i>
32	<i>to be measured</i>	<i>to be measured</i>	<i>calc</i>

Plus the qualitative TC3 checks: BGP session established (`birdc show protocols`), multiple equal-cost routes installed (`birdc show route`), per-uplink traffic balance (`ethtool -S p0 / p1`), and an explicit failover test (admin-shut one uplink while iperf3 is running, measure re-convergence).

7.6 Estimated time after the second fabric ask is in

Step	Wall clock
DPU-side: add second BGP neighbor to FRR on each DPU, verify two-path ECMP route installed	15 min
Deploy <code>dpf-ovn-hbn</code> cluster on existing 2 nodes	30 min
TC3 qualitative validation (BGP both peers Established, ECMP routes installed for two next-hops, per-uplink traffic balance, controlled failover)	30 min
TC4 11-test × 5-run matrix on HBN cluster	80 min
TC4 ECMP scaling sweep (1/2/4/8/16/32 parallel streams)	30 min
Report compilation (this doc + <code>STACK_EXPLANATION</code> + new charts)	15 min
Total	~3 hours

8. Limitations and Caveats

1. **n=4** per arm (runs 2–5; run 1 discarded as warmup per the test plan). Stdev is reported alongside the mean. For most metrics the deltas (+30 % to +290 %) are far larger than within-arm variance.
2. **Single-flow UDP loss is iperf3-bound, not network-bound.** The receiver's iperf3 process is the bottleneck; the same loss percentage appears in every arm because the bottleneck is identical. Multi-flow UDP with RSS would behave differently.
3. **Pods run on the DPU tenant cluster, not on the host cluster.** This was a forced choice — the host cluster runs Cilium and has no DPF acceleration path. Running the bench pods on the DPU tenant cluster keeps the dataplane on the DPU's br-int, which is the path under test. CPU costs reported are host-side (gpu1 and gpu2 hosts via the persistent `mpstat slicer`), not DPU-side, so the "host CPU saved" metric is the right one.
4. **Plumbing for VPC-OVN pod attachment was manual** (steps 6 & 7 in § 3.3). DPF v25.10.1's `vpc-ovn-node` agent only auto-binds OVS ports for pods with `ServiceInterface` CRs, not raw NAD-annotated pods. For production use the operator-managed flow should be used.
5. **HBN ECMP scaling can't be demonstrated yet — but the underlay is partially ready.** The first fabric ask (Eth1/24/26 routed /31s, BGP eBGP) is complete and validated end-to-end (BGP Established, 123 prefixes received per DPU, leaf loopback 11.0.0.111/32 reachable). The second fabric ask (additional BGP peering via VLAN 497's SVI so each DPU sees two equal-cost next-hops) is open with the network team — see § 7 and [FABRIC_HBN_SECOND_PEERING_REQUEST.md](#). Without the second path, BGP installs only one next-hop and the TC4 scaling sweep would read flat.
6. **Host PCIe link to BF3 is Gen3 x16 (8 GT/s), not Gen5.** The Supermicro X10 host pre-dates BF3 by ~5 years and its root complex tops out at PCIe Gen3. Theoretical PCIe bandwidth ~16 GB/s is well above the 40 GbE fabric requirement (~5 GB/s), so this does not bottleneck the test — but a newer host platform would enable Gen5 link-up. Worth recording for clients evaluating BF3 on newer hosts.

7. **The 40 GbE fabric speed is a switchport config choice, not a BF3 limit.** BF3 supports 200 GbE per port; the switch (Cisco Nexus, NX-OS 9.3, QSFP-H40G-AOC15M cables) is configured at 40 Gb/s. Numbers reported are at fabric line rate **as configured in this lab**, not at BF3's silicon ceiling.

9. Conclusion

DPF v25.10.1 VPC-OVN acceleration **delivers measurable, reproducible improvements** on every metric that exercises the host's networking path, on identical hardware:

- **Host CPU at line rate falls 76-77 %** — the DPU does the work that the host kernel does in passthrough mode.
- **Transaction rate (RR) rises ~2.7x, connection rate (CRR) rises ~3.9x** — conntrack and per-flow processing are hardware-offloaded.
- **Tail latency p99.9 drops 9x** (963 → 104 μ s) — predictable hardware path eliminates softirq jitter.
- **Throughput at the 40 Gb/s fabric line rate is identical** in both arms (39.4 Gbps with 8+ streams) — the wire is the limit; the win shows up in the CPU-busy column.
- **Single-stream TCP gains 33 %, UDP send rate gains 160 %** — TX-side offload is real.

The apples-to-apples Test Set 2 (same CNI, same cluster, hw-offload toggle) further confirms that **~half of those wins are silicon offload specifically** (the ConnectX-7 eswitch doing the per-packet work) and the other half comes from moving the dataplane to the DPU (OVS-DOCA on Arm cores still beats Cilium-on-host on transaction-heavy workloads).

For the customer-facing positioning ("the DPU pays for itself by freeing host cores while delivering better latency and connection scaling"), the data in this report supports it without qualification.

What's next: HBN/ECMP

The HBN/ECMP test (§ 7) adds a second value-prop — **multi-uplink aggregate bandwidth via BGP ECMP**. The fabric is **partially ready**:

-  The first fabric ask (Eth1/24/26 routed /31s, eBGP on each DPU's p1) is complete and validated. BGP is Established between each DPU and the leaf, 123 prefixes are propagating, and the leaf loopback 11.0.0.111/32 is reachable from each DPU at ~0.5 ms RTT.
-  The second fabric ask (a second BGP peering per DPU via VLAN 497's SVI, so the routing table sees two equal-cost next-hops) is open with the network team. Documented in [FABRIC_HBN_SECOND_PEEING_REQUEST.md](#). Without it, BGP installs only one path and the TC4 scaling sweep would be flat.

Once the second peering lands, the existing test infrastructure (scripts, OVN/NAD setup, runner) reuses cleanly for the `dpf-ovn-hbn` cluster — estimated ~3 hours wall clock to complete TC3 + TC4 + the ECMP scaling sweep.

Appendix A — File Index

results/	
└─ dpf-ovn-baseline/	Test Set 1 baseline – Cilium passthrough (n=4)
└─ dpf-ovn-accelerated/	Test Set 1 cluster B + Test Set 2 accelerated arm
(n=4)	
└─ dpf-ovn-accelerated-no-offload/	Test Set 2 baseline – VPC-OVN sw, hw-offload=false
(n=4)	
└─ charts/	generated PNGs (this report, 3-way comparisons)
scripts/	
└─ run_pod_baseline.sh	Test Set 1 cluster A runner
└─ run_pod_accelerated.sh	Test Set 1 cluster B + Test Set 2 accelerated runner
└─ run_pod_accelerated_no_offload.sh	Test Set 2 baseline (hw-offload=false) runner
└─ slice_host_mpstat.sh	cuts persistent host mpstat into per-test windows
└─ make_charts.py	generates the 3-way charts
Other reports / asks:	
└─ STACK_EXPLANATION.md	Full stack details (HW + SW + tunables + glue + data flow)
└─ FABRIC_HBN_ECMP_REQUEST.md	First fabric ask – COMPLETED (Eth1/24/26 routed /31s)
└─ FABRIC_HBN_SECOND_PEERING_REQUEST.md	Open follow-up ask (second BGP path per DPU via VLAN 497 SVI)
└─ POD_TO_POD_TEST_PLAN.md	Original test plan
└─ TEST_CASES.md	Higher-level TC1-TC4 scope (TC3/4 = HBN, pending)

Appendix B — Reproducing the Comparison

```
# 1. Deploy cluster A (passthrough) and run baseline
python3 scripts/dpf_deploy.py create dpf-ovn-baseline --hosts H1,H2
# wait for cluster Running, then:
bash scripts/run_pod_baseline.sh

# 2. Tear down, deploy cluster B (VPC-OVN), run accelerated
python3 scripts/dpf_deploy.py delete <baseline-uid>
python3 scripts/dpf_deploy.py create dpf-ovn-accelerated --hosts H1,H2
# wait for cluster Running and pods bound to OVN (see §3.3 for manual binding), then:
bash scripts/run_pod_accelerated.sh

# 3. Generate charts and report
python3 scripts/make_charts.py
```

All paths and IPs in the runner scripts are hard-coded for the lab inventory in [memory/env_details.md](#). Edit before running on a different inventory.